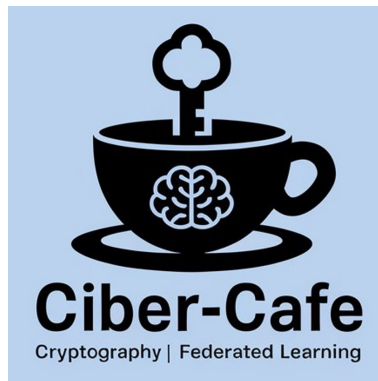




Universitat Jaume I

CIBER-CAFE Software Documentation

Technical Documentation

Date: March 6, 2025
Project: CIBER-CAFE
Version 1.0

Contents

Project Information	2
Disclaimer	2
Contact information	2
Introduction	3
Software Architecture	4
General Diagram	4
Directory Structure	5
Backend	7
Federated Learning	7
Homomorphic Encryption	8
Documentation	9
Homomorphic Encryption	9
EncrytionBase	9
TenSEAL	12
PyFHEL	16
OpenFHE	20
Registration	27
Federated Learning	28
FLClientBase	28
FLServerBase	29
Aggregate	30
Evaluate	31
Strategy	32
On Fit Config	34
Weighted Average	34

Project Information

ID: C121/23

FUNDING ENTITY: Instituto Nacional de Ciberseguridad de España

PROGRAMME: Proyectos Estratégicos de Ciberseguridad en España 2022

PERIOD: 01/01/2024 - 31/12/2025

NUMBER OF INVESTIGATORS: 8

Disclaimer

This software is a prototype and is currently under active development. It is provided for evaluation, testing, and experimentation purposes only. Future versions may introduce significant changes to its structure, functionality, performance, and features. As a result, this version may contain bugs, incomplete implementations, or experimental components that are subject to modification or removal.

Users should be aware that:

- The software may not be stable and is not recommended for use in production environments.
- Updates or future releases may not be backward compatible with this version.
- The functionality, interfaces, and dependencies may change without prior notice.
- Security, performance, and reliability optimizations are still in progress.

Users are encouraged to test and validate the software in a controlled environment and provide feedback for improvement. By using this software, you acknowledge and accept the risks associated with its prototype status.

Contact information

Name	Title	Contact information
Universitat Jaume I		
Manuel F. Dolz Zaragoza	-	Email: dolzm@uji.es
Sandra Catalán Pallarés	-	Email: catalans@uji.es
Darwin Quezada Gaibor	-	Email: quezada@uji.es

Introduction

This software is designed to implement Federated Learning (FL) with Homomorphic Encryption (HE), enabling secure, privacy-preserving, and decentralized model training across heterogeneous hardware architectures. It supports execution on CPUs and GPUs, leveraging hardware acceleration for optimized cryptographic operations and federated aggregation performance.

The framework provides modular integration with three cryptographic backends, including TenSEAL, OpenFHE, and PyFHEL, allowing users to perform encrypted computations using CKKS. It also supports Flower Federated Learning, enabling efficient communication, client coordination, and model aggregation across distributed environments.

Key features include:

- Encrypted weights with Homomorphic Encryption.
- Scalable federated learning workflows with configurable client participation and adaptive aggregation strategies.
- Customizable encryption contexts to balance security and computational efficiency.

This software is developed to facilitate secure collaborative learning in settings where raw data cannot be shared, making it suitable for applications on EDGE devices and the Cloud. By integrating privacy-preserving techniques with scalable federated architectures, it enables robust and secure AI model deployment in real-world scenarios.

Software Architecture

The software is structured to support Federated Learning (FL) with Homomorphic Encryption (HE), ensuring secure and decentralized model training. It is designed to be modular and adaptable, enabling execution across CPUs and GPUs while leveraging hardware acceleration for cryptographic and federated aggregation tasks.

General Diagram

The architecture described in the document is designed to support multiple cryptographic backends and federated learning frameworks, making it highly adaptable to different use cases and hardware platforms (see Fig. 1).

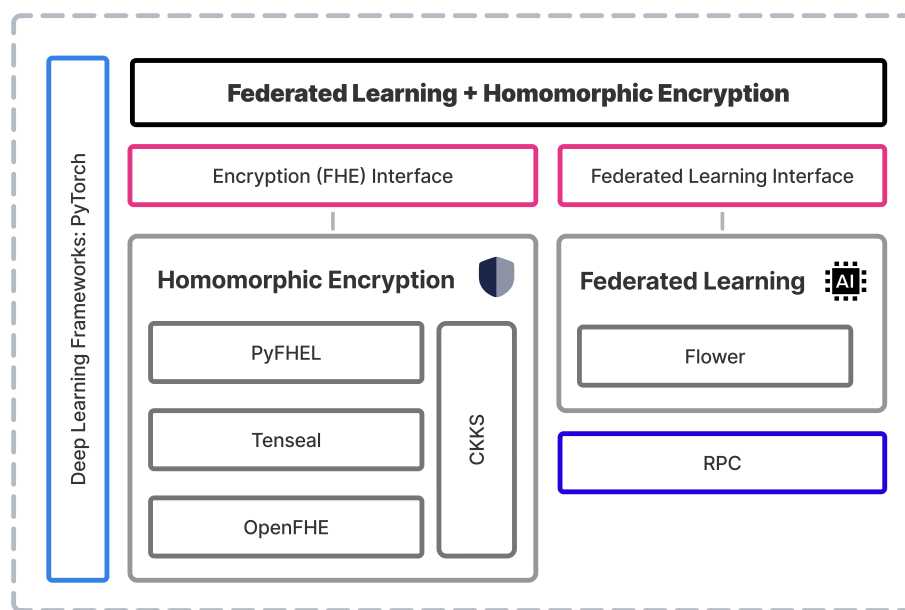


Figure 1: General FL+FHE architecture

Core Components

Federated Learning Layer

- Implements Federated Learning (FL) using the Flower framework.
- Provides a Federated Learning Interface to manage communication, client coordination, and model aggregation.
- Supports heterogeneous hardware, ensuring compatibility with various computing environments.

Homomorphic Encryption Layer

- Enables Fully Homomorphic Encryption (FHE) for secure computations on encrypted data.
- Supports multiple HE backends, and currently working with TenSEAL, PyFHE and OpenFHE.

Federated Learning + Homomorphic Encryption

- This software enables training of machine learning (ML) models using homomorphic encryption, specifically leveraging the TenSEAL library.
- By default, model weights are encrypted using homomorphic encryption before being shared between the server and clients and vice versa. However, users can configure the system to share weights without encryption if needed.

Deep Learning Framework

- The current prototype version supports only PyTorch.

Communication

- The current version supports Remote Procedure Call (RPC) protocol to communicate between the client and the server.

Directory Structure

```
CIBERCAFE/
|-- config/
|-- docker/
|   |-- client/
|   |   '-- Dockerfile
|   |-- server/
|   |   '-- Dockerfile
|-- example/
|   |-- encryption/
|   |   |-- test_pyphel.py
|   |   |-- test_tenseal.py
|   |   '-- test_openfhe.py
|   |-- models/
|   |   |-- custom_model.py
|   |   '-- quantum_model.py
|-- src/
|   |-- core/                                # Contains foundational components for the project
|   |   |-- exceptions/                      # Exception handling
|   |   |   |-- code_status.py
|   |   |   '-- core_exception.py
|   |   |-- typing/
|   |   |   '-- typing.py
|   |   |   util/                            #Common utilities
|   |   |   |-- check_value.py
|   |   |   |-- common.py
|   |   |   |-- data.py
|   |   |   |-- error.py
|   |   |   |-- plots.py
|   |   |   |-- load_config.py
|   |   |   '-- mapper.py
|   |-- encryption/                          # Encryption backends
|   |   |-- backends
```

```
| | | |-- openfhe/
| | | | '-- ckks.py
| | | |-- tenseal/
| | | | '-- ckks.py
| | | | '-- pyphel/
| | | | '-- ckks.py
| | |-- base.py
| | '-- factory.py
| |-- federated_learning/      # Federated learning functionality
| | |-- backends/             # Federated learning frameworks
| | | |-- flower/
| | | | |-- common/
| | | | | '-- flower_client.py
| | | | |-- server.py
| | | | '-- client.py
| | | |-- tenseal/
| | | | '-- ckks.py
| | | | '-- pyphel/
| | | | '-- ckks.py
| | |-- federation/           # Logic for Federated Learning
| | | |-- aggregate.py
| | | |-- evaluate.py
| | | |-- strategy.py
| | | |-- training_fit.py
| | | '-- weight_average.py
| | |-- client_base.py
| | |-- server_base.py
| | '-- factory.py
| |-- nn/                     # Neural network modules
| | |-- engine/
| | | |-- quantum.py
| | | |-- train.py
| | | '-- test.py
| | '-- layers/
| | | |-- encrypted_layer.py
| | | |-- layer.py
| | | '-- layer_util.py
| |-- storage/                # System's data storage
| | |-- data/
| | |-- he/
| | |-- models/
| | '-- stats/
|-- .flaskenv
|-- .gitignore
|-- config.py
|-- client.py
|-- server.py
```

```
|-- docker_compose.yml
|-- pyproject.toml
|-- pytest.py
|-- requirements.py
|-- run.py
'-- README.md
```

Backend

Federated Learning

Federated Learning operates in a hub-and-spoke model, where a central server coordinates multiple clients that participate in the training process. The server assigns tasks, gathers results, and aggregates model updates, while the clients execute computations locally and return results without sharing raw data.

A real-world FL deployment often involves multiple projects, each with unique hyperparameters, model architectures, aggregation strategies, or machine learning frameworks such as PyTorch. Thus, this software distinguishes two main components using the same denomination as Flower FL (see Fig. 2):

Flower Server Components

- *SuperLink*: A persistent component that manages communication between clients (SuperNodes) and their server, forwarding task instructions and receiving results.
- *ServerApp*: A short-lived process responsible for project-specific logic, including client selection, configuration, and result aggregation.

Flower Client Components

- *SuperNode*: A long-running process that connects to SuperLink, requests tasks, performs computations (e.g., local training), and sends results back.
- *ClientApp*: A transient component containing project-specific logic for training, evaluation, and data processing.

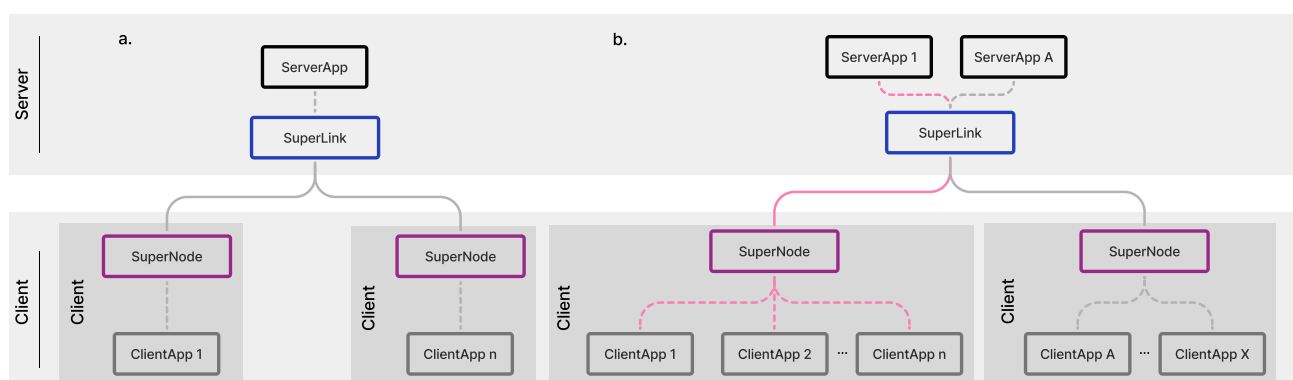


Figure 2: FL architecture. a) Hub-and-spoke. b) Multi-tenancy connection.

Fig. 2 illustrates two examples of federated learning under different architectures. In Fig. 2a, a hub-and-spoke model is shown, where a single *ServerApp* is connected to the *SuperLink*, with multiple *SuperNodes*, each hosting a single *ClientApp*. In contrast, Fig. 2b presents a multi-tenancy model, in which multiple *ServerApps* are connected to the *SuperLink*, and each *SuperNode* can host multiple *ClientApps*. This structure allows each *ServerApp* to support *ClientApp* from different projects simultaneously.

The *SuperNode* plays a crucial role as the primary computation unit, managing local training with decentralized datasets. Meanwhile, the *SuperLink* acts as the communication bridge, ensuring efficient coordination between clients and their server.

This modular architecture enhances scalability, flexibility, and interoperability, making this software adaptable for deploying federated learning across diverse environments. For further information about *SuperNodes* and *Superlink*, please check [Flower Federated Learning](#).

Homomorphic Encryption

The current prototype supports three Homomorphic Encryption (HE) libraries: TenSEAL, PyFHEL, and OpenFHE, all of which implement the CKKS encryption scheme for efficient computations on encrypted floating-point data. While all three libraries enable privacy-preserving operations, training neural networks on encrypted data is only supported with TenSEAL. This is because TenSEAL provides native support for encrypted tensors, allowing efficient manipulation of multidimensional data structures. Additionally, TenSEAL includes specialized operations such as `im2col` for Convolutional Neural Networks (CNNs), which facilitates encrypted convolution operations directly within its library, making it the only viable option for encrypted deep learning in this prototype.

Documentation

This section provides a detailed description of the functions implemented in the code, explaining their purpose, inputs, outputs, and interactions within the system. Each function plays an essential role in enabling federated learning, homomorphic encryption, and model training, ensuring seamless integration between different components. By analyzing the code structure and functionality, this section offers insights into how the prototype operates, how data flows between modules, and how key processes such as encryption, and model aggregation are handled.

Homomorphic Encryption

This section describes the code section related to homomorphic encryption.

EncryptionBase

The *EncryptionBase* class serves as an abstract base class (ABC) for implementing encryption frameworks. It defines a standard interface for different encryption backends such as OpenFHE, TenSEAL, PyFHE, ensuring that any subclass provides implementations for key management, encryption, decryption, and encrypted computations. This class is particularly used for integrating Homomorphic Encryption (HE) schemes in a federated learning system.

By using the ABC module and `@abstractmethod` decorators, this class enforces that all concrete implementations must define their own versions of these methods.

Class Definition

This is an abstract base class that establishes a framework for encryption methods.

```
class EncryptionBase(ABC):
```

Methods

```
@abstractmethod
def get_context(self):
    pass
```

- **Purpose:** Retrieves the encryption context, which holds all parameters necessary for encryption and decryption.
- **Returns:** The encryption context object.
- **Implementation:** Must be defined in subclasses to provide access to the specific encryption scheme's context.

```
@abstractmethod
def generate_keys(self, **kwargs):
    pass
```

- **Purpose:** Generates encryption keys based on the defined encryption scheme.

- **Parameters:**
 - ****kwargs** – Additional parameters for key generation.
- **Returns:** None.
- **Implementation:** Must be implemented in subclasses to create both public and private keys.

```
@abstractmethod
def get_private_key(self):
    pass
```

- **Purpose:** Retrieves the private key for decryption.
- **Returns:** The private key (secret key) object.
- **Implementation:** Must be implemented in subclasses to return the encryption scheme's private key.

```
@abstractmethod
def get_public_key(self):
    pass
```

- **Purpose:** Retrieves the public key used for encryption.
- **Returns:** The public key object.
- **Implementation:** Must be implemented in subclasses to return the encryption scheme's public key.

```
@abstractmethod
def encrypt(self, value, **kwargs):
    pass
```

- **Purpose:** Encrypts a given value using the encryption context.
- **Parameters:**
 - **value** – The value to encrypt.
 - ****kwargs** – Additional encryption parameters.
- **Returns:** The encrypted value.
- **Implementation:** Must be defined in subclasses to implement encryption logic.

```
@abstractmethod
def decrypt(self, value, **kwargs):
    pass
```

- **Purpose:** Decrypts an encrypted value using the encryption context.
- **Parameters:**
 - **value** – The encrypted value to decrypt.
 - ****kwargs** – Additional decryption parameters.
- **Returns:** The decrypted value.
- **Implementation:** Must be defined in subclasses to perform decryption.

```
@abstractmethod
def enc_dot(self, term1, term2, **kwargs):
    pass
```

- **Purpose:** Performs a dot product between two encrypted values.
- **Parameters:**
 - **term1** – The first encrypted operand.
 - **term2** – The second encrypted operand.
 - ****kwargs** – Additional computation parameters.
- **Returns:** The encrypted dot product result.
- **Implementation:** Must be implemented to define encrypted dot product logic.

```
@abstractmethod
def enc_add(self, term1, term2, **kwargs):
    pass
```

- **Purpose:** Adds two encrypted values.
- **Parameters:**
 - **term1** – The first encrypted value.
 - **term2** – The second encrypted value.
 - ****kwargs** – Additional computation parameters.
- **Returns:** The encrypted sum.
- **Implementation:** Must be implemented in subclasses to enable encrypted addition.

```
@abstractmethod
def enc_matmul(self, term1, term2, **kwargs):
    pass
```

- **Purpose:** Performs encrypted matrix multiplication.
- **Parameters:**
 - `term1` – The first encrypted matrix.
 - `term2` – The second encrypted matrix.
 - `**kwargs` – Additional computation parameters.
- **Returns:** The encrypted matrix multiplication result.
- **Implementation:** Must be implemented in subclasses to support encrypted matrix operations.

TenSEAL

Class Definition

This class implements encryption functionalities using the TenSEAL library for homomorphic encryption.

```
class TensealCKKS(EncryptionBase):
```

Attributes

- `poly_modulus_degree`: Polynomial modulus degree, default is 8192.
- `coef_mod_bit`: List of coefficient modulus bit sizes.
- `global_scale`: Global scaling factor for CKKS.
- `context`: Stores the encryption context.

Methods

```
def create_context(self):  
    pass
```

- **Purpose:** Initializes and configures the encryption context using CKKS scheme.
- **Returns:** None.
- **Implementation:** Sets polynomial modulus degree, coefficient modulus bit sizes, and global scale.

```
def set_context(self, context):  
    pass
```

- **Purpose:** Sets an existing encryption context.

- Parameters:
 - context: Serialized encryption context.
- Returns: A deserialized encryption context.

```
def set_poly_modulus_degree(self, degree=8192):  
    pass
```

- Purpose: Updates the polynomial modulus degree.
- Parameters:
 - degree (int): The polynomial modulus degree, must be a power of 2.
- Returns: None.

```
def set_coef_mod_bit(self, coefficient=None):  
    pass
```

- Purpose: Sets the coefficient modulus bit sizes for encryption.
- Parameters:
 - coefficient (list of int): Coefficient modulus bit sizes.
- Returns: None.

```
def set_global_scale(self, scale=2**40):  
    pass
```

- Purpose: Sets the global scale used for encryption.
- Parameters:
 - scale (int or float): The scaling factor for encrypted values.
- Returns: None.

```
def set_private_key(self, private_key):  
    pass
```

- Purpose: Sets the private key for encryption.
- Parameters:
 - private_key: The serialized private key.
- Returns: None.

```
def set_public_key(self, public_key):  
    pass
```

- **Purpose:** Sets the public key for encryption.
- **Parameters:**
 - **public_key:** The serialized public key.
- **Returns:** None.

```
def plain_tensor(self, value):  
    pass
```

- **Purpose:** Converts a numerical tensor into a plaintext tensor.
- **Parameters:**
 - **value:** A numerical array or list.
- **Returns:** A TenSEAL plaintext tensor.

```
def encrypt(self, value):  
    pass
```

- **Purpose:** Encrypts numerical arrays, values or tensors using the CKKS encryption scheme.
- **Parameters:**
 - **value:** A numpy array, list, or numpy matrix containing the data to be encrypted.
- **Returns:** An encrypted CKKS vector or CKKS tensor.
- **Implementation:** Converts the input into a compatible format and encrypts it using the TenSEAL library.

```
def decrypt(self, value):  
    pass
```

- **Purpose:** Decrypts an encrypted value.
- **Parameters:**
 - **value:** A CKKS encrypted object (vector or tensor).
- **Returns:** The decrypted plaintext values.

```
def plain_vector(self, value):  
    pass
```

- **Purpose:** Encodes a plain vector into a format suitable for encrypted operations.
- **Parameters:**
 - **value:** A numpy array representing the plaintext data.
- **Returns:** An encoded plaintext vector.

```
def enc_dot(self, term1, term2):  
    pass
```

- **Purpose:** Computes the dot product of two encrypted vectors or tensors.
- **Parameters:**
 - **term1:** A CKKS vector or CKKS tensor.
 - **term2:** A CKKS vector or CKKS tensor.
- **Returns:** The encrypted dot product of the inputs.

```
def enc_add(self, term1, term2):  
    pass
```

- **Purpose:** Performs element-wise addition of two encrypted vectors or tensors.
- **Parameters:**
 - **term1:** A CKKS vector or CKKS tensor.
 - **term2:** A CKKS vector or CKKS tensor.
- **Returns:** The encrypted sum of the two inputs.

```
def enc_matmul(self, term1, term2):  
    pass
```

- **Purpose:** Performs matrix multiplication of an encrypted vector/tensor with a plaintext tensor.
- **Parameters:**
 - **term1:** A CKKS vector or CKKS tensor.
 - **term2:** A plaintext tensor.
- **Returns:** The encrypted matrix multiplication result.

Example

```
he_backend = HomomorphicEncryptionFactory.
```

```
get_backend(library='TENSEAL', schema='CKKS')
he_backend.set_poly_modulus_degree(8192)
he_backend.generate_keys()

vector = np.array([1.5, 2.6, 3.8])
encrypted_vector = he_backend.encrypt(vector)
dencrypted_vector = he_backend.decrypt(encrypted_vector)
print(dencrypted_vector)

matrix = np.matrix([[1.1, 2.2], [3.3, 4.4]])
encrypted_matrix = he_backend.encrypt(matrix)
dencrypted_matrix = he_backend.decrypt(encrypted_matrix)
print(dencrypted_matrix)

vector1 = np.array([1.5, 2.6, 3.8])
vector2 = np.array([1.5, 2.6, 3.8])

encrypted_vector1 = he_backend.encrypt(vector1)
encrypted_vector2 = he_backend.encrypt(vector2)
dot_product = he_backend.enc_dot(encrypted_vector1, encrypted_vector2)
dencrypted_vector = he_backend.decrypt(dot_product)

matrix1 = np.matrix([[1.1, 2.2], [3.3, 4.4]])
matrix2 = np.matrix([[1.1, 2.2], [3.3, 4.4]])

encrypted_matrix1 = he_backend.encrypt(matrix1)
encrypted_matrix2 = he_backend.encrypt(matrix2)
dot_matrix = he_backend.enc_dot(encrypted_matrix1, encrypted_matrix2)
dencrypted_dot_matrix = he_backend.decrypt(dot_matrix)
print(dencrypted_dot_matrix)

matrix = np.matrix([[1.1, 2.2], [3.3, 4.4]])
vector = np.array([1.5, 2.6])

plain_matrix = he_backend.plain_tensor(matrix)
encrypted_vector = he_backend.encrypt(vector)
matmul = he_backend.enc_matmul(encrypted_vector, plain_matrix)
dencrypted_matmul = he_backend.decrypt(matmul)
print(dencrypted_matmul)
```

PyFHEL

Class Definition

This class implements homomorphic encryption using the CKKS scheme through the PyFHE library.

```
class PyFHELCKKS(EncryptionBase):
```

Attributes

- `poly_modulus_degree`: Polynomial modulus degree, default is 8192.
- `coef_mod_bit`: List of coefficient modulus bit sizes.
- `global_scale`: Global scaling factor for CKKS.
- `context`: Stores the encryption context.
- `he`: Instance of the Pyfhel encryption engine.

Methods

```
def create_context(self):
```

- **Purpose**: Initializes the encryption context with CKKS parameters.
- **Parameters**: None.
- **Returns**: None.
- **Implementation**: Generates an encryption context using PyFHEL.

```
def get_context(self):
```

- **Purpose**: Retrieves the encryption context.
- **Parameters**: None.
- **Returns**: The encryption context object.

```
def get_instance(self):
```

- **Purpose**: Returns the current instance of the object.
- **Returns**: The current instance (`self.he`).

```
def set_context(self, context):
```

- **Purpose**: Sets the context for the object.
- **Parameters**:

- context: The context to be loaded.

- Returns: None.

```
def set_poly_modulus_degree(self, degree=8192):
```

- Purpose: Sets the polynomial modulus degree for the context.

- Parameters:

- degree: The polynomial modulus degree (default is 8192).

- Returns: None.

```
def set_coef_mod_bit(self, coefficient=None):
```

- Purpose: Sets the coefficient modulus bit sizes for the context.

- Parameters:

- coefficient: A list of integers representing the coefficient modulus bit sizes (optional).

- Returns: None.

```
def set_global_scale(self, scale=2**40):
```

- Purpose: Sets the global scale for the context.

- Parameters:

- scale: The global scale (default is 2^{40}).

- Returns: None.

```
def generate_keys(self):
```

- Purpose: Generates keys for the homomorphic encryption scheme.

- Returns: None.

```
def rotate_key_gen(self):
```

- Purpose: Generates the rotation key for the homomorphic encryption scheme.

- Returns: None.

```
def set_private_key(self, private_key):
```

- Purpose: Sets the private key for the homomorphic encryption scheme.

- Parameters:
 - `private_key`: The private key (in `bytes` format).
- Returns: None.

```
def set_public_key(self, public_key):
```

- Purpose: Sets the public key for the homomorphic encryption scheme.
- Parameters:
 - `public_key`: The public key (in `bytes` format).
- Returns: None.

```
def get_private_key(self):
```

- Purpose: Gets the serialized private key.
- Returns: The serialized private key.

```
def get_public_key(self):
```

- Purpose: Gets the public key in `bytes` format.
- Returns: The public key in `bytes` format.

```
def encrypt(self, value):
```

- Purpose: Encrypts a scalar, vector, or matrix.
- Parameters:
 - `value` (`int`, `float`, `list`, `np.array`, `np.matrix`) - The plaintext data to encrypt.
- Returns: The encrypted ciphertext.
- Implementation: Uses PyFHE to encrypt numerical values.

```
def decrypt(self, value):
```

- Purpose: Decrypts an encrypted ciphertext.
- Parameters:
 - `value` (`PyCtxt`, `list`, `array`, `matrix`) The encrypted ciphertext to decrypt.

- **Returns:** The decrypted plaintext value.
- **Implementation:** Uses Pyfhel decryption method.

```
def enc_dot(self, term1, term2):
```

- **Purpose:** Computes the encrypted dot product of vectors or matrices.
- **Parameters:**
 - `term1, term2` (PyCtxt, np.ndarray) - Encrypted inputs.
- **Returns:** The encrypted dot product result.

```
def enc_matmul(self, term1, term2):
```

- **Purpose:** Performs homomorphic matrix multiplication.
- **Parameters:**
 - `term1, term2` (PyCtxt, np.ndarray) - Encrypted matrix and vector.
- **Returns:** The encrypted matrix multiplication result.

OpenFHE

Class Definition

This class implements homomorphic encryption using the CKKS scheme through the OpenFHE library.

```
class OpenFHECKKS(EncryptionBase):
```

Attributes

- `poly_modulus_degree`: Polynomial modulus degree, default is 8192.
- `coef_mod_bit`: List of coefficient modulus bit sizes.
- `global_scale`: Global scaling factor for CKKS.
- `secret_key_dist`: Distribution type for the secret key.
- `security_level`: Security level for encryption.
- `ring_dim`: Ring dimension used in encryption.
- `first_mod_size`: Size of the first modulus.
- `context`: Stores the encryption context.

- **scaling_technique:** Technique used for scaling encrypted values.
- **parameters:** Stores CKKS encryption parameters.
- **keys:** Stores encryption keys.

Methods

```
def create_context(self):
```

- **Purpose:** Initializes the encryption context with CKKS parameters.
- **Parameters:** None.
- **Returns:** None.
- **Implementation:** Generates an encryption context using OpenFHE.

```
def set_secret_key_dist(self, secret_key_dist):
```

- **Purpose:** Sets the distribution type of the secret key.
- **Parameters:** `secret_key_dist` - The distribution type for the secret key (UNIFORM_TERNARY, SPARSE_TERNARY and GAUSSIAN).
- **Returns:** None.
- **Implementation:** Updates the secret key distribution in encryption parameters.

```
def set_ring_dim(self, ring_dim):
```

- **Purpose:** Sets the ring dimension for encryption.
- **Parameters:** `ring_dim` - The ring dimension value.
- **Returns:** None.
- **Implementation:** Updates the ring dimension in encryption parameters.

```
def set_first_mod_size(self, first_mod_size):
```

- **Purpose:** Sets the first modulus size.
- **Parameters:** `first_mod_size` - The size of the first modulus.
- **Returns:** None.
- **Implementation:** Updates the first modulus size in encryption parameters.

```
def set_scaling_technique(self, scaling_technique):
```

- **Purpose:** Sets the scaling technique for CKKS encryption.
- **Parameters:** `scaling_technique` - The scaling technique to use (FIXEDMANUAL, FIXEDAUTO, FLEXIBLEAUTO, FLEXIBLEAUTOEXT, NORESALE and INVALID_RS_TECHNIQUE).
- **Returns:** None.
- **Implementation:** Updates the scaling technique in encryption parameters.

```
def set_global_scale(self, global_scale):
```

- **Purpose:** Sets the global scaling factor for CKKS encryption.
- **Parameters:** `global_scale` - The global scaling value.
- **Returns:** None.
- **Implementation:** Updates the scaling modulus size in encryption parameters.

```
def set_security_level(self, security_level):
```

- **Purpose:** Sets the security level for encryption.
- **Parameters:** `security_level` - The security level to use (HESTD_128_CLASSIC, HESTD_192_CLASSIC, HESTD_256_CLASSIC and HESTD_NOTSET).
- **Returns:** None.
- **Implementation:** Updates the security level in encryption parameters.

```
def create_context(self):
```

- **Purpose:** Initializes the encryption context using the CKKS parameters.
- **Parameters:** None.
- **Returns:** None.
- **Implementation:** Creates an instance of `CKKSEncryptionContext` with the specified parameters.

```
def set_depth(self, depth):
```

- **Purpose:** Sets the multiplicative depth for encryption operations.
- **Parameters:** `depth` - The desired multiplicative depth.
- **Returns:** None.
- **Implementation:** Updates the encryption parameters to set the multiplicative depth.

```
def set_context_feature(self, context_feature: PKESchemaFeature):
```

- **Purpose:** Enables a specific encryption feature in the context.
- **Parameters:** `context_feature` - The feature to enable (PKE, KEYSWITCH, PRE, LEVELED SHE, ADVANCED SHE, MULTIPARTY, FHE and SCHEMESWITCH).
- **Returns:** None.
- **Implementation:** Calls the `Enable` function on the encryption context with the given feature.

```
def eval_bootstrap_setup(self, level_budget, bsgs_dim, num_slots):
```

- **Purpose:** Sets up bootstrapping parameters for encrypted computation.
- **Parameters:** `level_budget` - The level budget for bootstrapping.
- `bsgs_dim` - The baby-step giant-step dimension.
- `num_slots` - The number of slots in plaintext space.
- **Returns:** None.
- **Implementation:** Calls `EvalBootstrapSetup` on the encryption context with the given parameters.

```
def make_ckks_packed_plain_text(self, value, size: int = None, level: int = None, params=None, slots: int = None):
```

- **Purpose:** Creates a CKKS packed plaintext from the given values.
- **Parameters:** `value` - The data to encode.
- `size, level, params, slots` - Optional CKKS encoding parameters.
- **Returns:** The encoded plaintext object.
- **Implementation:** Converts input values into CKKS packed plaintexts using the encryption context.

```
def eval_mult_key_gen(self):
```

- **Purpose:** Generates multiplication keys for encrypted arithmetic operations.
- **Parameters:** None.
- **Returns:** None.
- **Implementation:** Calls `EvalMultKeyGen` on the encryption context.

```
def get_context(self):
```

- **Purpose:** Retrieves the encryption context.
- **Parameters:** None.
- **Returns:** The encryption context object.
- **Implementation:** Returns the stored encryption context instance.

```
def generate_keys(self, **kwargs):
```

- **Purpose:** Generates encryption keys for secure operations.
- **Parameters:** None.
- **Returns:** None.
- **Implementation:** Calls `KeyGen` on the encryption context to generate keys.

```
def get_private_key(self):
```

- **Purpose:** Retrieves the private decryption key.
- **Parameters:** None.
- **Returns:** The secret key.
- **Implementation:** Returns the `secretKey` from the generated keys.

```
def get_public_key(self):
```

- **Purpose:** Retrieves the public encryption key.
- **Parameters:** None.
- **Returns:** None (Function not implemented).
- **Implementation:** Currently a placeholder function.

```
def encrypt(self, value, **kwargs):
```

- **Purpose:** Encrypts a given value.
- **Parameters:** `value` - Data to encrypt (numpy array, list, or matrix).
- **Returns:** Encrypted ciphertext.
- **Implementation:** Converts input into encrypted format using the encryption context.

```
def decrypt(self, value, **kwargs):
```

- **Purpose:** Decrypts an encrypted value.
- **Parameters:** `value` - Encrypted ciphertext to decrypt.
- **Returns:** Decrypted plaintext.
- **Implementation:** Calls decryption method on the encrypted value.

```
def enc_dot(self, term1, term2, **kwargs):
```

- **Purpose:** Computes encrypted dot product for vectors and matrices.
- **Parameters:**
 - `term1` - First encrypted vector or matrix.
 - `term2` - Second encrypted vector or matrix.
- **Returns:** Encrypted result of the dot product.
- **Implementation:** Calls appropriate vector or matrix multiplication method.

```
def vec_dot(self, term1, term2):
```

- **Purpose:** Computes encrypted dot product for vectors.
- **Parameters:** Two encrypted vectors of the same length.
- **Returns:** Encrypted scalar result.
- **Implementation:** Iteratively multiplies and sums elements.

```
def vec_matrix_dot(self, term1, term2):
```

- **Purpose:** Computes encrypted vector-matrix multiplication.
- **Parameters:**
 - `term1` - Encrypted vector.
 - `term2` - Encrypted matrix.
- **Returns:** Encrypted vector result.
- **Implementation:** Iterates over columns, performing element-wise multiplication and summation.

```
def matrix_dot(self, term1, term2):
```

- **Purpose:** Computes encrypted matrix-matrix multiplication.
- **Parameters:** Two encrypted matrices with compatible dimensions.
- **Returns:** Encrypted matrix result.
- **Implementation:** Iterates over rows and columns, performing element-wise multiplication and summation.

```
def enc_add(self, term1, term2, **kwargs):
```

- **Purpose:** Computes encrypted addition for vectors and matrices.
- **Parameters:**
 - **term1** - First encrypted vector or matrix.
 - **term2** - Second encrypted vector or matrix.
- **Returns:** Encrypted result of the addition.
- **Implementation:** Calls appropriate vector or matrix addition method.

```
def enc_matmul(self, term1, term2, **kwargs):
```

- **Purpose:** Computes encrypted matrix multiplication.
- **Parameters:** Two encrypted matrices or vectors.
- **Returns:** Encrypted matrix or vector result.
- **Implementation:** Calls **enc_dot** method.

```
def vec_dot(self, term1, term2):
```

- **Purpose:** Computes encrypted dot product for vectors.
- **Parameters:** Two encrypted vectors of the same length.
- **Returns:** Encrypted scalar result.
- **Implementation:** Iteratively multiplies and sums elements.

```
def vec_matrix_dot(self, term1, term2):
```

- **Purpose:** Computes encrypted vector-matrix multiplication.
- **Parameters:**
 - **term1** - Encrypted vector.

- **term2** - Encrypted matrix.
- **Returns:** Encrypted vector result.
- **Implementation:** Iterates over columns, performing element-wise multiplication and summation.

```
def matrix_dot(self, term1, term2):
```

- **Purpose:** Computes encrypted matrix-matrix multiplication.
- **Parameters:** Two encrypted matrices with compatible dimensions.
- **Returns:** Encrypted matrix result.
- **Implementation:** Iterates over rows and columns, performing element-wise multiplication and summation.

Registration

Class Definition

This class serves as a factory for creating instances of homomorphic encryption backends, supporting multiple encryption libraries and schemas.

```
class HomomorphicEncryptionFactory:
```

Methods

```
@classmethod
def register_backend(cls, library, schema, backend_class):
    cls._backend[(library, schema)] = backend_class
```

- **Purpose:** Registers a backend class for a specific encryption library and schema.
- **Parameters:**
 - **library:** The name of the encryption library (e.g., 'TENSEAL').
 - **schema:** The encryption schema (e.g., 'CKKS').
 - **backend_class:** The class implementing the backend.
- **Returns:** None.
- **Implementation:** Stores the backend class in a dictionary mapping (library, schema) to backend classes.

```
@classmethod
def get_backend(cls, library='TENSEAL', schema='CKKS'):
    backend_cls = cls._backend.get((library, schema))
    if not backend_cls:
        raise ValueError(f"Unsupported library 'library' or schema 'schema'")
    return backend_cls()
```

- **Purpose:** Retrieves the registered backend class for a given library and schema.
- **Parameters:**
 - **library:** The encryption library (default: 'TENSEAL').
 - **schema:** The encryption schema (default: 'CKKS').
- **Returns:** An instance of the registered backend class.
- **Implementation:** Looks up the backend dictionary and raises an error if the requested library and schema are not supported.

Example Usage

```
factory = HomomorphicEncryptionFactory.get_backend(library='TENSEAL',
schema='CKKS') encrypted_backend = factory()
```

Federated Learning

This section describes the code section related to Federated Learning.

FLClientBase

The *FLClientBase* class serves as an abstract base class (ABC) for implementing the client-side logic in a federated learning system. It defines an abstract method, `client`, which must be implemented by any subclass. This method is responsible for managing the client-specific operations, including training and validation of the model, data handling, and interacting with the homomorphic encryption (HE) scheme.

The method's parameters, such as `cid`, `trainloader`, `valloader`, and others, represent various aspects of the client's workflow, such as the client ID, data loaders, and paths for saving results. The `device` and `batch_size` parameters help configure the computational environment, while the `he` parameter indicates the homomorphic encryption enable or not.

By using the ABC module and `@abstractmethod` decorators, this class ensures that all concrete implementations must provide their own version of the `client` method, enforcing a consistent interface for client-side federated learning operations.

Class Definition This class defines the base framework for Federated Learning (FL) clients.

```
class FLClientBase(ABC):
```

```
@abstractmethod def client(self, cid, trainloader, valloader,
    device, batch_size, save_results, plot_path,
    roc_path, yaml_path, he, classes, context_client):
    pass
```

Methods

- **Purpose:** Defines the method that each FL client must implement for training and validation.
- **Parameters:**
 - `cid`: Client ID.
 - `trainloader`: Data loader for training.
 - `valloader`: Data loader for validation.
 - `device`: Computational device (CPU/GPU).
 - `batch_size`: Size of each training batch.
 - `save_results`: Boolean flag to save results.
 - `plot_path`: Path for saving plots.
 - `roc_path`: Path for saving ROC curve data.
 - `yaml_path`: Path for YAML configuration file.
 - `he`: Use or not Homomorphic encryption.
 - `classes`: Number of output classes.
 - `context_client`: Encryption context for the client.
- **Returns:** None (must be implemented in subclasses).
- **Implementation:** This method must be implemented by all subclasses to handle client-side federated learning operations.

FLServerBase

This class serves as an abstract base class (ABC) for implementing server-side operations in a federated learning system. It defines an abstract method `server` that must be implemented by any subclass. The method will manage server-side logic, such as aggregating model updates from clients and handling the necessary computations.

```
class FLServerBase(ABC):
```

Methods

```
def server(self, func, **kwargs):
```

- **Purpose:** The method to be implemented by subclasses, which will define the server's operations, such as aggregating model updates from clients. The method will also handle other server-side operations like parameter synchronization and potential application of homomorphic encryption schemes.
- **Parameters:** - **func:** A function that will be used to perform the server-side operation. - ****kwargs:** Additional keyword arguments that may vary depending on the specific implementation of the server's operations.
- **Returns:** The result of the server-side operation, typically involving aggregated model updates or computations.
- **Implementation:** Ensures that subclasses provide a concrete implementation of the server-side federated learning process.

Aggregate

This function handles the aggregation of model parameters from multiple federated learning clients. It takes aggregated parameters, converts them into a format compatible with the central model, and updates the model state accordingly. The function also allows for saving aggregated parameters to disk and handling checkpoints if needed.

```
def aggregate(server_round, aggregated_parameters, central_model,
              checkpoint=None, context_client=None, context_server=None, server_path=None):
```

Parameters

- **server_round:** The current round of server-side aggregation.
- **aggregated_parameters:** A collection of parameters that have been aggregated from the client models.
- **central_model:** The central model on the server that will be updated with the aggregated parameters.
- **checkpoint:** (Optional) The path where the model checkpoint will be saved.
- **context_client:** (Optional) Context information related to the client, used when saving or serializing the server response.
- **context_server:** The context information on the server, used for serializing the aggregated parameters.
- **server_path:** The file path where server-related data (such as aggregated parameters) will be written.

Purpose

This function is responsible for:

Aggregating the model parameters from federated clients and updating the central model. Saving the aggregated parameters and the model's state to a file. Handling optional checkpoint saving for the model.

Implementation

If the `aggregated_parameters` are not `None`, the function: Converts the aggregated parameters into NumPy arrays using `parameters_to_ndarrays_custom`. If `context_client` is provided, the function serializes both the server context and the aggregated model parameters and saves them to a specified path (`server_path`). If `context_client` is not provided, it directly updates the `central_model`'s state dictionary with the aggregated parameters. If the `checkpoint` is provided, the function saves the central model's state as a checkpoint in the specified directory. Returns The function doesn't return any value but performs file writes for saving aggregated parameters and checkpoint states.

Evaluate

This function performs the evaluation of a federated learning model on the server side. It takes the aggregated parameters from the federated clients, updates the model, and computes the loss and accuracy on the test dataset.

```
def evaluate_fl(server_round: int, parameters: NDArray, config: Dict[str,
Scalar], testloader, device, model_init) -> Optional[Tuple[float, Dict[str,
Scalar]]:
```

Parameters

- `server_round`: The current round of the federated learning process (used for logging or other tracking purposes).
- `parameters`: The aggregated model parameters (in the form of an NDArray) that will be applied to the initialized model.
- `config`: A dictionary containing configuration settings (such as hyperparameters or other settings) used during the evaluation process.
- `testloader`: The data loader that provides the test dataset for evaluation.
- `device`: The computing device (CPU or GPU) where the model will be evaluated.
- `model_init`: The model object that will be initialized with the provided parameters before evaluation.

Purpose The function performs the evaluation of the federated learning model by:

- Setting the provided parameters on the initialized model.
- Computing the loss and accuracy on the test dataset.
- Returning the evaluation results (loss and accuracy).

Implementation

The function begins by calling `set_parameters` to apply the given aggregated parameters to the model (`model_init`). The `CrossEntropyLoss` function is initialized for calculating the loss during evaluation. It then uses the `test` function to evaluate the model on the test dataset, calculating the loss, accuracy, and other metrics. The loss and accuracy are printed to the console for logging purposes. The function returns the loss value and a dictionary containing the accuracy.

Returns

loss: The loss computed during the evaluation of the model on the test data. A dictionary containing the **accuracy** metric.

Strategy

Class Definition

This class implements a custom federated learning strategy by extending `fl.server.strategy.Strategy`. It allows for fine-grained control over client selection, training, evaluation, and aggregation processes.

```
class FedCustom(fl.server.strategy.Strategy):
```

Methods

```
def initialize_parameters(self, client_manager: ClientManager) ->
Optional[Parameters]:
```

- **Purpose:** Initializes the global model parameters.
- **Parameters:**
 - **client_manager:** The client manager handling client connections.
- **Returns:** Initial global model parameters.
- **Implementation:** Returns the predefined initial parameters and prevents storing them in memory.

```
def num_fit_clients(self, num_available_clients: int) -> Tuple[int, int]:
```

- **Purpose:** Determines the number of clients to use for federated training.
- **Parameters:**
 - **num_available_clients:** The total number of available clients.
- **Returns:** Tuple containing the sample size and minimum required clients.

```
def configure_fit(self, server_round: int, parameters: Parameters,
client_manager: ClientManager) -> List[Tuple[ClientProxy, FitIns]]:
```

- **Purpose:** Configures the next round of training by sampling clients and defining learning parameters.
- **Parameters:**
 - **server_round:** The current round of federated learning.
 - **parameters:** The current global model parameters.
 - **client_manager:** The client manager handling client connections.
- **Returns:** List of client-configuration pairs for training.

```
def aggregate_fit(self, server_round: int, results: List[Tuple[ClientProxy, FitRes]], failures: List[Union[Tuple[ClientProxy, FitRes], BaseException]]) -> Tuple[Optional[Parameters], Dict[str, Scalar]]:
```

- **Purpose:** Aggregates model updates received from clients using weighted averaging.
- **Parameters:**
 - **server_round:** The current training round.
 - **results:** A list of successful client training results.
 - **failures:** A list of failed training attempts.
- **Returns:** Aggregated model parameters and computed metrics.

```
def configure_evaluate(self, server_round: int, parameters: Parameters, client_manager: ClientManager) -> List[Tuple[ClientProxy, EvaluateIns]]:
```

- **Purpose:** Configures the next round of evaluation by selecting clients.
- **Parameters:**
 - **server_round:** The current evaluation round.
 - **parameters:** The current global model parameters.
 - **client_manager:** The client manager handling client connections.
- **Returns:** List of client-configuration pairs for evaluation.

```
def aggregate_evaluate(self, server_round: int, results: List[Tuple[ClientProxy, EvaluateRes]], failures: List[Union[Tuple[ClientProxy, EvaluateRes], BaseException]]) -> Tuple[Optional[float], Dict[str, Scalar]]:
```

- **Purpose:** Aggregates evaluation results from clients using weighted averaging.
- **Parameters:**

- **server_round**: The current evaluation round.
- **results**: A list of successful client evaluation results.
- **failures**: A list of failed evaluation attempts.
- **Returns**: Aggregated evaluation loss and computed metrics.

On Fit Config

```
def on_fit_config(epoch=2, lr=0.001, batch_size=32) -> Callable[[int], Dict[str, str]]:
```

- **Purpose**: Generates a configuration dictionary for federated training.
- **Parameters**:
 - **epoch**: Number of local training epochs.
 - **lr**: Learning rate for training.
 - **batch_size**: Size of training batches.
- **Returns**: A callable function that provides training configurations per round.
- **Implementation**: Returns a function that takes the server round as input and outputs a dictionary with training configurations.

Weighted Average

```
def weighted_average(metrics: List[Tuple[int, Metrics]]) -> Metrics:
```

- **Purpose**: Computes the weighted average of client metrics.
- **Parameters**:
 - **metrics**: A list of tuples, each containing the number of examples and a dictionary of metric values.
- **Returns**: A dictionary with the aggregated accuracy.
- **Implementation**: Weights the accuracy of each client by the number of examples and computes the overall weighted average.